Actividad DaC y Teorema Maestro

Delgado Villagrana Aurora

El teorema maestro sirve para calcular la complejidad de ciertos algoritmos recursivos que pueden expresarse de la siguiente forma:

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

Donde:

- a: número de llamadas recursivas.
- b: tamaño de cada subproblema.
- d: exponente en el tiempo de ejecución del “paso de combinación”

Para aplicar el teorema maestro, comparamos $T(n)$ con:

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad [Case\ 1] \\ O(n^d) & \text{if } a < b^d \quad [Case\ 2] \\ O(n^{\log_b a}) & \text{if } a > b^d \quad [Case\ 3]. \end{cases}$$

Si el algoritmo no tiene la estructura requerida, no se puede aplicar directamente el teorema maestro.

1. **Búsqueda binaria**

La búsqueda binaria requiere que los datos estén ordenados. En cada paso compara el elemento buscado con el elemento central y descarta la mitad del arreglo.

Su recurrencia es:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Identificamos los valores:

- a = 1
- b = 2
- d = 0

Calculamos:

$$b^{\mathrm{d}} = 2^0 = 1$$

$$a = b^d \quad \Rightarrow \quad 1 = 1$$

Por lo tanto, aplicamos el caso 1:

$$T(n) = O(n^0 \log n)$$

$$\boxed{T(n) = O(\log n)}$$

2. **Multiplicación recursiva de matrices**

Consideramos la multiplicación tradicional por bloques, que divide las matrices en cuatro partes y realiza ocho multiplicaciones recursivas.

La recurrencia es:

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2)$$

Identificamos los valores:

- a = 8
- b = 2
- d = 2

Calculamos:

$$b^d = 2^2 = 4$$

$$a > b^d \quad \Rightarrow \quad 8 > 4$$

Por lo tanto, aplicamos el caso 3:

$$T(n) = O\left(n^{\log_b a}\right)$$

$$T(n) = O\left(n^{\log_2 8}\right)$$

$$\boxed{T(n) = O(n^3)}$$

3. **Multiplicación de matrices por el método de Strassen**

Strassen realiza siete multiplicaciones recursivas de matrices más pequeñas.

La recurrencia es:

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

Identificamos los valores:

- a = 7
- b = 2
- d = 2

Calculamos:

$$b^d = 2^2 = 4$$

$$a > b^d \quad \Rightarrow \quad 7 > 4$$

Por lo tanto, aplicamos el caso 3:

$$T(n) = O\left(n^{\log_b a}\right)$$

$$T(n) = O\left(n^{\log_2 7}\right)$$

$$\boxed{T(n) = O(n^{2.807})}$$

4. **Números de Fibonacci**

Para el algoritmo recursivo sencillo, la recurrencia es:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

No tiene la forma requerida por el teorema maestro y no podemos identificar a y b como pide la fórmula.

5. **Encontrar el par de números más cercanos**

El algoritmo de divide y vencerás divide los datos en dos mitades, resuelve cada mitad y combina los resultados.

La recurrencia es:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Identificamos los valores:

- a = 2
- b = 2
- d = 1

Calculamos:

$$b^d = 2^1 = 2$$

$$a = b^d \quad \Rightarrow \quad 2 = 2$$

Por lo tanto, aplicamos el caso 1:

$$T(n) = O(n^d \log n)$$

$$\boxed{T(n) = O(n \log n)}$$

6. Búsqueda en un árbol binario de búsqueda

A. Árbol binario de búsqueda ordenado y balanceado

En un árbol balanceado, la búsqueda continúa por una sola rama y el tamaño del problema se reduce aproximadamente a la mitad.

La recurrencia es:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Identificamos los valores:

- a = 1
- b = 2
- d = 0

Calculamos:

$$b^d = 2^0 = 1$$

$$a = b^d \quad \Rightarrow \quad 1 = 1$$

Por lo tanto, aplicamos el caso 1:

$$T(n) = O(n^d \log n)$$

$$\boxed{T(n) = O(\log n)}$$

B. Árbol binario sin ordenamiento

Cuando el árbol no está ordenado, no sabemos en cuál rama está el elemento. En el peor caso debemos recorrer todos los nodos.

Una recurrencia para recorrer los nodos es:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

Identificamos los valores:

- a = 2
- b = 2
- d = 0

Calculamos:

$$b^d = 2^0 = 1$$

$$a > b^d \quad \Rightarrow \quad 2 > 1$$

Por lo tanto, aplicamos el caso 3:

$$T(n) = O\left(n^{\log_b a}\right)$$

$$T(n) = O\left(n^{\log_2 2}\right)$$

$$\boxed{T(n) = O(n)}$$